Assignment 2 (Questions): (Knowing your data, filtering, and sorting

Q1. Load the (SuperMarket Analysis.csv) dataset and display the first 5 rows

In [3]:
```
import pandas as pd

df = pd.read_csv("SuperMarket_Analysis.csv")
df.head()
```

Out[3]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415
1	226-31-3081	Giza	Naypyitaw	Normal	Female	Electronic accessories	15.28	5	3.8200
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085

Q2. Display the dataset shape (rows and columns).

In [4]:
```
df.shape
```

Out[4]:
```
(1000, 17)
```

Q3. List all column names.

In [5]:
```
df.columns.tolist()
```

Out[5]:
```
['Invoice ID',
 'Branch',
 'City',
 'Customer type',
 'Gender',
 'Product line',
 'Unit price',
 'Quantity',
 'Tax 5%',
 'Sales',
 'Date',
 'Time',
 'Payment',
 'cogs',
 'gross margin percentage',
 'gross income',
 'Rating']
```

Q4. Identify categorical and numerical columns.

In [6]:
```
categorical_cols = df.select_dtypes(include="object").columns.tolist()
numerical_cols = df.select_dtypes(include="number").columns.tolist()

print("Categorical columns:", categorical_cols)
print("Numerical columns:", numerical_cols)
```

```
Categorical columns: ['Invoice ID', 'Branch', 'City', 'Customer type', 'Gender',
'Product line', 'Date', 'Time', 'Payment']
Numerical columns: ['Unit price', 'Quantity', 'Tax 5%', 'Sales', 'cogs', 'gross m
argin percentage', 'gross income', 'Rating']
```

```
D:\anaconda7\tmp\ipykernel_15304\833685544.py:1: Pandas4Warning: For backward com
patibility, 'str' dtypes are included by select_dtypes when 'object' dtype is spe
cified. This behavior is deprecated and will be removed in a future version. Expl
icitly pass 'str' to `include` to select them, or to `exclude` to remove them and
silence this warning.
See https://pandas.pydata.org/docs/user_guide/migration-3-strings.html#string-mig
ration-select-dtypes for details on how to write code that works with pandas 2 an
d 3.
  categorical_cols = df.select_dtypes(include="object").columns.tolist()
```

Q5. Check for missing values in each column

In [7]:
```
df.isnull().sum()
```

Out[7]:
```
Invoice ID                0
Branch                    0
City                      0
Customer type             0
Gender                    0
Product line              0
Unit price                0
Quantity                  0
Tax 5%                    0
Sales                     0
Date                      0
Time                      0
Payment                   0
cogs                      0
gross margin percentage   0
gross income              0
Rating                    0
dtype: int64
```

Q6. Display the data types of each column.

In [8]:
```
df.dtypes
```

Out[8]:
```
Invoice ID                 str
Branch                     str
City                       str
Customer type              str
Gender                     str
Product line               str
Unit price             float64
Quantity                 int64
Tax 5%                 float64
Sales                  float64
Date                       str
Time                       str
Payment                    str
cogs                   float64
gross margin percentage float64
gross income           float64
Rating                 float64
dtype: object
```

Q7. Show summary statistics for numerical columns (use pandas method)

In [9]:
```
df.describe()
```

Out[9]:

	Unit price	Quantity	Tax 5%	Sales	cogs	gross margin percentage	
count	1000.000000	1000.000000	1000.000000	1000.000000	1000.00000	1000.000000	10
mean	55.672130	5.510000	15.379369	322.966749	307.58738	4.761905	
std	26.494628	2.923431	11.708825	245.885335	234.17651	0.000000	
min	10.080000	1.000000	0.508500	10.678500	10.17000	4.761905	
25%	32.875000	3.000000	5.924875	124.422375	118.49750	4.761905	
50%	55.230000	5.000000	12.088000	253.848000	241.76000	4.761905	
75%	77.935000	8.000000	22.445250	471.350250	448.90500	4.761905	
max	99.960000	10.000000	49.650000	1042.650000	993.00000	4.761905	

Q8. Filter rows where Sales > 500.

In [10]:
```
high_sales = df[df["Sales"] > 500]
high_sales.head()
```

Out[10]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.141
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.208
5	699-14-3026	Giza	Naypyitaw	Member	Female	Electronic accessories	85.39	7	29.886
7	315-22-5665	Giza	Naypyitaw	Member	Female	Home and lifestyle	73.56	10	36.780
14	829-34-3910	Alex	Yangon	Member	Female	Health and beauty	71.38	10	35.690

Q9. Filter sales in City = "Yangon" and Sales > 200.

In [11]:
```
yangon_high_sales = df[(df["City"] == "Yangon") & (df["Sales"] > 200)]
yangon_high_sales.head()
```

Out[11]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%	
0	750-67-8428	Alex	Yangon	Member	Female	Health and beauty	74.69	7	26.1415	5
2	631-41-3108	Alex	Yangon	Normal	Female	Home and lifestyle	46.33	7	16.2155	3
3	123-19-1176	Alex	Yangon	Member	Female	Health and beauty	58.22	8	23.2880	4
4	373-73-7910	Alex	Yangon	Member	Female	Sports and travel	86.31	7	30.2085	6
6	355-53-5943	Alex	Yangon	Member	Female	Electronic accessories	68.84	6	20.6520	4

Q10. Sort all orders by Sales in descending order

In [12]:
```
sorted_by_sales = df.sort_values(by="Sales", ascending=False)
sorted_by_sales.head()
```

Out[12]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Tax 5%
350	860-79-0874	Giza	Naypyitaw	Member	Female	Fashion accessories	99.30	10	49.65
167	687-47-8271	Alex	Yangon	Normal	Male	Fashion accessories	98.98	10	49.49
557	283-26-5248	Giza	Naypyitaw	Member	Female	Food and beverages	98.52	10	49.26
699	751-41-9720	Giza	Naypyitaw	Normal	Male	Home and lifestyle	97.50	10	48.75
996	303-96-2227	Cairo	Mandalay	Normal	Female	Home and lifestyle	97.38	10	48.69

Q11. Sort by Date (ascending) and then Time (ascending)

In [13]:
```
df["Date"] = pd.to_datetime(df["Date"])
sorted_by_date_time = df.sort_values(by=["Date", "Time"], ascending=[True, True]
sorted_by_date_time.head()
```

Out[13]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Ta 5%
17	765-26-6951	Alex	Yangon	Member	Female	Sports and travel	72.61	6	21.78
970	746-04-1077	Cairo	Mandalay	Member	Female	Food and beverages	84.63	10	42.31
839	271-77-8740	Giza	Naypyitaw	Member	Female	Sports and travel	29.22	6	8.76
523	133-14-7229	Giza	Naypyitaw	Normal	Male	Health and beauty	62.87	2	6.28
567	651-88-7328	Alex	Yangon	Normal	Female	Fashion accessories	65.74	9	29.58

Q12. Sort by Unit price and Quantity

In [14]:
```
sorted_by_price_qty = df.sort_values(by=["Unit price", "Quantity"], ascending=[T
sorted_by_price_qty.head()
```

Out[14]:

	Invoice ID	Branch	City	Customer type	Gender	Product line	Unit price	Quantity	Ta 5%
944	333-23-2632	Alex	Yangon	Member	Male	Health and beauty	10.08	7	3.528
572	239-48-4278	Alex	Yangon	Member	Male	Food and beverages	10.13	7	3.545
784	516-77-6464	Giza	Naypyitaw	Member	Female	Health and beauty	10.16	5	2.540
822	784-21-9238	Giza	Naypyitaw	Member	Male	Sports and travel	10.17	1	0.508
881	115-38-7388	Giza	Naypyitaw	Member	Female	Fashion accessories	10.18	8	4.072

Q13. Calculate the total sales in "Sales" per Branch

In [15]:
```
total_sales_per_branch = df.groupby("Branch")["Sales"].sum()
total_sales_per_branch
```

Out[15]:
```
Branch
Alex      106200.3705
Cairo     106197.6720
Giza      110568.7065
Name: Sales, dtype: float64
```

Q14. Calculate average Sales per City.

In [16]:
```
avg_sales_per_city = df.groupby("City")["Sales"].mean()
avg_sales_per_city
```

Out[16]:
```
City
Mandalay      319.872506
Naypyitaw     337.099715
Yangon        312.354031
Name: Sales, dtype: float64
```

Q15. Find the quantities sold per product line

In [17]:
```
qty_per_product_line = df.groupby("Product line")["Quantity"].sum()
qty_per_product_line
```

Out[17]:
```
Product line
Electronic accessories    971
Fashion accessories       902
Food and beverages        952
Health and beauty         854
Home and lifestyle        911
Sports and travel         920
Name: Quantity, dtype: int64
```

Q16. Calculate average gross income per Gender.

In [18]:
```
avg_gross_income_per_gender = df.groupby("Gender")["gross income"].mean()
avg_gross_income_per_gender
```

Out[18]:
```
Gender
Female    16.234829
Male      14.240749
Name: gross income, dtype: float64
```

Q17. Count number of sales per Payment method

In [19]:
```
sales_count_per_payment = df["Payment"].value_counts()
sales_count_per_payment
```

Out[19]:
```
Payment
Ewallet        345
Cash           344
Credit card    311
Name: count, dtype: int64
```

Q18. Find maximum Sales per Branch

In [20]:
```
max_sales_per_branch = df.groupby("Branch")["Sales"].max()
max_sales_per_branch
```

Out[20]:
```
Branch
Alex     1039.29
Cairo    1022.49
Giza     1042.65
Name: Sales, dtype: float64
```

Q19. Find minimum Unit price per Product line.

In [21]:
```
min_unit_price_per_product_line = df.groupby("Product line")["Unit price"].min()
min_unit_price_per_product_line
```

Out[21]:
```
Product line
Electronic accessories    10.56
Fashion accessories       10.18
Food and beverages        10.13
Health and beauty         10.08
Home and lifestyle        10.53
Sports and travel         10.17
Name: Unit price, dtype: float64
```

Q20. Find the sum of gross income per Product line and Branch

In [22]:
```
gross_income_per_line_branch = df.groupby(["Product line", "Branch"])["gross inc
gross_income_per_line_branch
```

Out[22]:
```
Product line            Branch
Electronic accessories  Alex       872.2435
                        Cairo      811.9735
                        Giza       903.2845
Fashion accessories     Alex       777.7385
                        Cairo      781.5865
                        Giza      1026.6700
Food and beverages      Alex       817.2905
                        Cairo      724.5185
                        Giza      1131.7550
Health and beauty       Alex       599.8930
                        Cairo      951.4600
                        Giza       791.2060
Home and lifestyle      Alex      1067.4855
                        Cairo      835.6745
                        Giza       661.6930
Sports and travel       Alex       922.5095
                        Cairo      951.8190
                        Giza       750.5680
Name: gross income, dtype: float64
```

---

21. What is the total quantities sold in Product line: "Electronic accessories"?

In [23]:
```
total_qty_electronics = df[df["Product line"] == "Electronic accessories"]["Quan
total_qty_electronics
```

Out[23]:
```
np.int64(971)
```

---

22. What is the average Sales for female customers?

In [24]:
```
avg_sales_female = df[df["Gender"] == "Female"]["Sales"].mean()
avg_sales_female
```

Out[24]:
```
np.float64(340.9314141856392)
```

---

23. What is the most expensive Unit price among Customer type members only?

In [25]:
```
max_unit_price_members = df[df["Customer type"] == "Member"]["Unit price"].max()
max_unit_price_members
```

Out[25]:
```
np.float64(99.96)
```

---

24. How many orders with Rating >= 9 ?

In [26]:
```
orders_high_rating = df[df["Rating"] >= 9].shape[0]
orders_high_rating
```

Out[26]:
```
166
```

---

25. What is the total Sales for Payment " Credit card " in Branch C?

In [27]:
```
total_sales_credit_card_branchC = df[(df["Payment"] == "Credit card") & (df["Bra
total_sales_credit_card_branchC
```

Out[27]:
```
np.float64(0.0)
```

In [ ]:
```

```